PIM requirements coverage

Assignment user stories

The supplied implementation covers US1-US11 in Appendix B of the COMP3211 Course Project Description 2026. The table maps each story to code and automated checks. Test paths below are relative to tests/. Related cases may also exercise the same story.

Story	Implemented behavior	Main code	Verification
US1	Create and manage four PIR types in one collection	records.py; manager.py	test_records: test_all_types_and_fields_are_preserved
US2	Create plain-text notes	Note; PIM.create	test_records: test_all_types_and_fields_are_preserved
US3	Create tasks with description and deadline	Task; parse_time	test_records: test_all_types_and_fields_are_preserved
US4	Create events with description, start and alarm	Event; parse_time	test_records: alarm validation; test_criteria: each time field
US5	Create contacts with name, address and mobile	Contact	test_records: all types; test_storage: contact roundtrip
US6	Update existing fields atomically	Record.updated; PIM.update	test_records: test_updates_are_immutable_and_validate_all_changes_together
US7	Type/text/time search; nested and/or/negation	criteria.py; PIM.search	test_criteria: every text/time field, type, operators and precedence
US8	Print details of one or all PIRs	Controller; view.terminal	test_commands: test_complete_session
US9	Delete a specified PIR	PIM.delete	test_manager: test_list_get_update_delete_and_ids_never_reused
US10	Save all PIRs to a .pim file	storage.save	test_storage: roundtrip; failed writes; extension checks
US11	Load all PIRs from a .pim file	storage.load; from_snapshot	test_storage: roundtrip; malformed/duplicate data; test_commands: failed load

All main code names in the table refer to the model package unless explicitly identified as controller or view.

Implemented requirement rules

No group SRS was supplied. The identifiers below describe this implementation and are proposed traceability identifiers, not claims about an unseen SRS. Replace or map them to the group's final SRS identifiers before submission.

ID	Verifiable rule	Status and evidence
IR1	Every PIR has a positive stable ID and one of four types	Implemented; records and manager tests
IR2	All type-specific text and time fields are required; blank text is rejected	Implemented; records tests

ID	Verifiable rule	Status and evidence
IR3	Time inputs include date/time, normalize to Hong Kong, and alarm is no later than start	Implemented; records and criteria tests
IR4	Update failure preserves the original record; type and ID are immutable	Implemented; records tests
IR5	All five text fields support case-insensitive contains	Implemented; criteria tests
IR6	All three time fields support strict <, > and instant equality	Implemented; criteria tests
IR7	Search supports parentheses, negation, conjunction and disjunction with documented precedence	Implemented; criteria tests
IR8	Invalid criteria are rejected even in an empty collection	Implemented; criteria tests
IR9	Display/search output contains record IDs, types and all fields	Implemented; command tests
IR10	Persistence round trips every type and preserves the next ID after deletions	Implemented; storage tests
IR11	Invalid files cannot partially replace the active collection	Implemented; storage and command tests
IR12	Failed atomic replacement preserves an existing destination	Implemented; simulated storage failure test
IR13	Unsaved changes block unforced load and quit	Implemented; command tests
IR14	Application and model tests use Python standard libraries only	Implemented; source imports and dependency-free test commands
IR15	Domain, validation, search and persistence code are in model	Implemented; directory structure and one-way imports
IR16	Expected command errors are reported and the interactive session continues	Implemented; command-session tests

Interpretation and deliverable boundaries

Alarms are stored and searchable timestamps. Background notifications are not a user story in the supplied Appendix B and are not implemented. There is no GUI, networking, shared database, recurring event scheduler or automatic save. These are outside the supplied functional scope.

The coding package includes source, automated tests, developer/user manuals, this coverage mapping and a measured model line coverage report. It does not complete the full SRS, formal design diagrams, demonstration MP4, presentation materials or signed Honour Declaration. No claim is made that the complete group submission is ready.

OpenAI Codex assisted with the implementation, tests and documentation. The group should acknowledge that actual usage in the Honour Declaration and record its own reviewed contribution percentages, names, IDs, signatures and dates.